EcommerceMultiVendor — Architecture

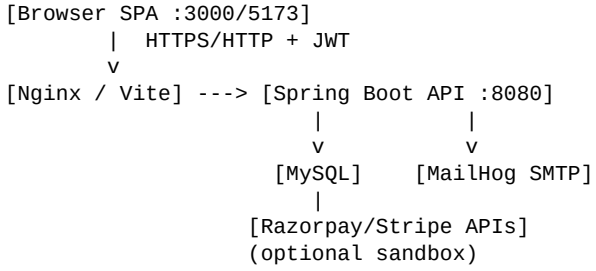
1. Goals

BazaarHub is a multi-vendor marketplace where customers browse and buy products from approved seller shops, sellers manage inventory and fulfillment, and admins govern categories, coupons, and seller approval. A rule-based chatbot answers operational questions (orders, cart, catalog, FAQs) without an LLM.

2. Stack rationale

• **Spring Boot 4 / Java 21**: mature security, JPA, mail, and REST ecosystem for marketplace domains.
• **MySQL**: relational model fits users, shops, products, carts, orders, and payments.
• **JWT + Spring Security**: stateless API auth for SPA clients; role checks for CUSTOMER / SELLER / ADMIN.
• **React + TypeScript + Redux Toolkit**: typed SPA with predictable auth/cart state.
• **MUI + Tailwind**: component library for forms/dashboards plus utility styling.
• **Formik/Yup**: validated login/register flows.
• **Recharts**: seller and admin summary charts.
• **Razorpay + Stripe SDKs**: sandbox payment intents with mock fallback when keys are placeholders.
• **MailHog / Java Mail**: local SMTP capture for order and approval emails.
• **Docker Compose**: one-command local stack (MySQL, mail, API, nginx SPA).

3. High-level architecture



Packages live under com.ecommerce.multivendor: domain, repository, service, controller, security, chatbot, config, exception.

4. Auth flow

1. Client posts email/password to /api/v1/auth/login (or register).
2. AuthService authenticates via AuthenticationManager and issues HS256 JWT (JwtService).

3. Subsequent requests send Authorization: Bearer <token>.

4. JwtAuthenticationFilter loads AuthenticatedUser and sets the SecurityContext.

5. Method/URL rules enforce role scopes (/api/v1/seller/**, /api/v1/admin/**).

5. Checkout and payment flow

1. Customer adds items to cart and saves an address.

2. POST /api/v1/checkout creates an OrderEntity (PENDING), decrements stock, applies optional coupon, clears cart, emails confirmation.

3. POST /api/v1/payments creates a Payment for RAZORPAY, STRIPE, or COD.

4. Sandbox/mock references are returned when keys are placeholders; POST /payments/confirm marks payment CAPTURED and order PAID.

5. Sellers update status to SHIPPED / DELIVERED; customers may cancel PENDING/PAID orders.

6. Chatbot design

IntentMatcher classifies messages with keywords and regex (order id, category, product query). ChatbotService maps intents to existing domain services and returns { intent, reply }. Personal data requires JWT. No external LLM keys.

7. Entity relationship (overview)

• User 1—1 Shop (seller); Shop 1—* Product; Product *—1 Category

• User 1—1 Cart —* CartItem —1 Product

• User *—* Product via WishlistItem

• User 1—* Address; User 1—* Order —* OrderItem —1 Product/Shop

• Order 1—1 Payment; Order *—0..1 Coupon

• Product 1—* Review

8. Deployment topology

Compose services: mysql, mailhog, backend (JRE image + fat JAR), frontend (nginx serving Vite dist and proxying /api to backend). Local Gradle/npm workflows mirror the same API contract.

9. Security notes

• Passwords hashed with BCrypt.

• CORS restricted to configured origins.

- Admin self-registration blocked.
- Seller product APIs require an approved shop.
- Chatbot refuses seller/admin summary intents for other roles.